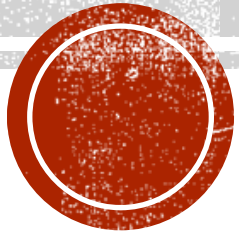
Faculty of Economics and Business Administration

Introduction to Dart programming – PART 1



Dr. Abdul Rahman EL SAYED

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

ERROR HANDLING

- Modern programming languages use try-catch blocks to deal with exceptions (errors) that may arise at runtime. Below is an example

```
bool flag = true;
do {
  try {
    stdout.write('Enter x: ');
    int x = int.parse(stdin.readLineSync()!);
    stdout.write('Enter y: ');
    int y = int.parse(stdin.readLineSync()!);
    print('Sum is: ${x + y}');
    flag = false;
  } catch (e) {
    print('Only numbers are allowed');
  }
} while (flag);
```

TOPICS

- ERROR HANDLING
- **FUNCTIONS**
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

FUNCTIONS

- Functions in Dart are declared in a similar way to java. Below is an example to calculate the factorial of a number using a function.

```
int factorial(int x) {
  int f = 1;
  for (int i = 2; i <= x; i++) {
    f *= i;
  }
return f;
}

void main() {
  print('Factorial of 10 is ${factorial(10)}');
}
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- **OVERLOADING FUNCTIONS**
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

OVERLOADING FUNCTIONS

- Dart doesn't support function overloading. Although, you can emulate function overloading via optional paramaters. You place optional parameters inside [ ]. In the below code , y is an optional parameter with a default value of 0.

```
int f1(int x, [int y = 0]) {
  return x * 2 + y;
}

void main() {
  print('f(10) ${f1(10)}');
  print('f(10, 20) ${f1(10, 20)}');
}
```

OVERLOADING FUNCTIONS

- You can also define optional function parameters without default values. In this case it will have a default value of null. You also need to declare the variable using int?, since Dart diesn't allow null values by default.

```
int f1(int x, [int? y]) {
  if (y != null) return x * 2 + y;
  return x * 2;
}

void main() {
  print('f(10) result is: ${f1(10)}');
  print('f(10, 20) result is: ${f1(10, 20)}');
}
```

```
Output:
f(10) result is: 20
f(10, 20) result is: 40
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- **NAMED FUNCTION PARAMETERS**
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

NAMED FUNCTION PARAMETERS

- You can also define named function parameters by enclosing them inside { }. This way you can define their values using their names. They are automatically defined as optional parameters, so you have to give them default values. You can define them in any order you wish.

```
int f1({int x = 0, int y = 0}) {
  return x + y * 3;
}

void main() {
  print('f1() result is: ${f1()}'); // output: 0
  print('f1(x: 10) result is: ${f1(x: 10)}'); // output: 10
  print('f1(y:10) result is: ${f1(y: 10)}'); // output: 30
  // when using named parameters, the order does not matter.
  print('f1(x:5, y:6) result is: ${f1(x: 5, y: 10)}'); // output: 35
  print('f1(y:6, x:5) result is: ${f1(y: 10, x: 5)}'); // output: 35
}
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- **LAMBDA FUNCTIONS**
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

LAMBDA FUNCTIONS

- You can use lambda functions, if your function contains only a single statement. You use the => to write a lambda function. You don't have to write a return statement, as the expression is returned automatically.

```
int sum(int x, int y) => x + y;
void display(String name) => print('Hello $name');

void main() {
  print('sum(10, 20) result is: ${sum(10, 20)}');
  display('Jane');
}
```

Output:

```
sum(10, 20) result is: 30
Hello Jane
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- **FUNCTION TYPE**
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

FUNCTION TYPE

- In Dart, Function is a type, like String or num. This means that they can also be assigned to fields or local variables or passed as parameters to other functions; consider the following example:

```
String sayHello() {
  return "Hello world!";
}

void main() {
  var sayHelloFunction = sayHello; // assigning the function
  // to the variable
  print(sayHelloFunction()); // prints Hello world!
}
```

Output:

```
Hello world!
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- **ANONYMOUS FUNCTIONS**
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

ANONYMOUS FUNCTIONS

- Anonymous functions are functions that do not have a name. You can use them to pass a function as a parameter to another function. The type Function is used to declare a variable type. Function(int) means this function takes a single parameter of type int.

```
// you can declare a parameter as a function type
void display(Function(int) f, int y) {
  print('Result: ${f(y) + y}');
}

void main() {
  // you can write the function body when you pass it to the function
  display((int x) {
    return x * x;
  }, 10);
}
```

Output:

```
Result: 110
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- **FIXED-SIZE LISTS**
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

FIXED-SIZE LISTS

- Lists in Dart are similar to arrays in other languages. You can create fixed length and variable length Lists. Below is an example of creating a fixed size list and printing its contents.

```
// create an initialized fixed size list of type String
List<String> colors = ['red', 'green', 'blue'];
// print its contents using a for loop
print('List items using for loop...');
for (int i = 0; i < colors.length; i++) {
  print('${colors[i]}');
}
// printing items using anonymous function
// the List class forEach method, takes an anonymous function
// as parameter and executes once for each element in the list
print('List items using forEach method...');
colors.forEach((e) {
  print('$e');
});
```

FIXED-SIZE LISTS

- You can create an empty fixed-size list using the named constructor filled. It takes 2 parameters (size, fill value).

```
// create a fixed-size empty list of size 4 and type double
// all elements in the list are initialized to 0
List<double> grades = List.filled(4, 0);
// add four elements to the list
grades[0] = 80;
grades[1] = 60;
grades[2] = 40;
grades[3] = 90;
// print passed grades only
print('Passed grades...');
grades.forEach((e) {
  if (e >= 60) print('$e');
});
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- **DYNAMIC LISTS**
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

DYNAMIC LISTS

- You can create an empty dynamic list using the [] literals. You can use the add method, to add elements to a dynamic list.

```
// create a variable-size empty list of type int
List<int> numbers = [];
// add four elements to the list
numbers.add(6);
numbers.add(9);
numbers.add(1);
numbers.add(4);
// print even numbers only
print('Even numbers...');
numbers.forEach((e) {
  if (e % 2 == 0) print('$e');
});
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- **COMBINING LISTS**
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

COMBINING LISTS

- You can use the addAll method to combine Lists, as show in the below example.

```
  main() {
// Creating lists
   List l1 = ['Welcome', 'to'];
   List l2 = ['CSCI410'];

// Combining the elements of l2 with l1
   l1.addAll(l2);

// Printing combined list
   print(l1);
}
```

Output:

```
[Welcome, to, CSCI410]
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- **MAPS**
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

MAPS

- The Map object is a simple key/value pair. Keys and values in a map may be of any type. A Map is a dynamic collection. In other words, Maps can grow and shrink at runtime. Below is an example of creating a map using literals { }.

```
void main() {
// create a map using literals
var user = {'id': '101', 'name': 'rami'};
print('Id: ${user['id']}');
print('Name: ${user['name']}');
// you can also define the type of the key/value pairs
var exams = <String, double>{'exam1': 80, 'exam2': 60, 'final': 77};
print(exams);
}
```

```
Output:
Id: 101
Name: rami
{exam1: 80.0, exam2: 60.0, final: 77.0}
```

MAPS

- You can create an empty map using the Map constructor. Below we will create a map that holds student's ids and grades. We can then get a student's grade by providing an id.

```
void main() {
  // create a map using Map constructor
  var students = new Map<int, double>();
  students[101] = 70;
  students[201] = 50;
  students[303] = 80;
  students[400] = 77.5;

  print('Student with id 303 grade is: ${students[303]}');
}
```

```
Output:
Student with id 303 grade is: 80.0
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- **GENERICS AND DART LITERALS**
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

GENERICS AND DART LITERALS

- If you check out this chapter's list and map examples, you will see we used the [] and {} literals to initialize them. With generics, we can specify a type during the initialization, adding a <elementType>[] prefix for lists and <keyType, elementType>{} for maps.

```
main() {
  var avengerNames = <String>["Hulk", "Captain America"];
  var avengerQuotes = <String, String>{
    "Captain America": "I can do this all day!",
    "Spider Man": "Am I an Avenger?",
    "Hulk": "Smaaaaaash!"
  };
  print('List: $avengerNames');
  print('Map: $avengerQuotes');
}
```

```
Output:
List: [Hulk, Captain America]
Map: {Captain America: I can do this all day!, Spider Man: Am I an Avenger?, Hulk: Smaaaaaash!}
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- **SETS**
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

DART SETS

- Sets in Dart is a special case in List where all the inputs are unique i.e it doesn't contain any repeated input. It can also be interpreted as an unordered array with unique inputs.

```
main() {
  final characters1 = <String>{'A', 'B', 'C'};
  final characters2 = <String>{'A', 'E', 'B'};
  // declare a set that contains unique values.
  // The intersection method will return a unique set
  final Set<String> intersectionSet = characters1.intersection(characters2);
  print(intersectionSet);
}
```

Output:

```
{A, B}
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- **PASSING LISTS TO FUNCTIONS**
- FUNCTIONS THAT RETURN LISTS
- PACKAGES

PASSING LISTS TO FUNCTIONS

- The below code shows how you can pass a list to a function using literals.

```
ouble getAverage(List<int> numbers) {
  int sum = 0;
  numbers.forEach((e) {
    sum += e;
  });
  return sum / numbers.length; // no need for typecasting, the / will return double value
}

void main() {
  double avg = getAverage([50, 60, 70]);
  print('Average is: $avg');
}
```

Output:

```
Average is: 60.0
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- **FUNCTIONS THAT RETURN LISTS**
- PACKAGES

FUNCTIONS THAT RETURN LISTS

- You can write a function to return a list. Below is an example for a function that takes a list of grades and returns another list containing only passed grades.

```
List<double> getPassed(List<double> grades) {
  List<double> passed = <double>[];
  grades.forEach((e) {
    if (e >= 60) passed.add(e);
  });
  return passed;
}
void main() {
  var passed = getPassed([50, 60, 70]);
  print('Passed grades...');
  passed.forEach((e) {
    print('$e');
  });
}
```

TOPICS

- ERROR HANDLING
- FUNCTIONS
- OVERLOADING FUNCTIONS
- NAMED FUNCTION PARAMETERS
- LAMBDA FUNCTIONS
- FUNCTION TYPE
- ANONYMOUS FUNCTIONS
- FIXED-SIZE LISTS
- DYNAMIC LISTS
- COMBINING LISTS
- MAPS
- GENERICS AND DART LITERALS
- SETS
- PASSING LISTS TO FUNCTIONS
- FUNCTIONS THAT RETURN LISTS
- **PACKAGES**

PACKAGES

- Dart has many packages which extends the functionality of the language. One example is the math package, which includes many useful mathematical functions.

```
import 'dart:math';

main() {
  double x = 2;
  double y = 25;
  // use pow function to return x^y
  print('$x to the power $y is ${pow(x, y)}');
  // use the sqrt function to return the square root of  number
  print('Square root of $y is ${sqrt(y)}');
}
```

Output:

```
2.0 to the power 25.0 is 33554432.0
Square root of 25.0 is 5.0
```